Compensation Plan

Calculation Logic Extraction

A developer reference containing only the formulas, percentages, thresholds and decision rules needed to build the compensation engine — extracted from the Aglotrends Business Plan & Compensation Plan document.

SOURCE
Aglotrends Business Plan & Compensation Plan (Sections 7, 9–16)

PREPARED FOR
Engineering / Backend Implementation

PRODUCT
Aglotrends Platform

REVISION
2026-10-10 · rebranded (green → blue)

Compliance note — read before building. This plan is *recruitment-funded*: 55% of every registration fee is redistributed as network bonuses. Whether that is lawful depends entirely on jurisdiction and on how the product is presented. Securities-law classification, AML/KYC, data privacy (NDPR), and the Member Agreement are **out of scope of this extraction** and must be confirmed with counsel before go-live.

CONTENTS

1. Global Constants
2. Registration Fee Allocation
3. Direct Referral Bonus
4. Generation Bonus (Gen 2–6)
5. Rank Bonus Pool & Associate Rank Milestones
6. Leadership Bonus Pool & Leadership Ranks
7. Trading Profit Sharing
8. Matrix Placement & Spillover Logic
9. Active Member, Compression & Rank Maintenance
10. Reserve / Sustainability Model
11. Worked Examples (Test Cases)
12. Consolidated Reconciliation Check

SECTION 1

Global Constants

These are the fixed inputs the entire compensation engine is built on. All other calculations derive from these two numbers (fee and funding floor) plus the structural constants.

Constant	Value	Notes
`REGISTRATION_FEE`	**$20.00 (USD)**	One-time fee paid by every new member on signup.
`MIN_FUNDING`	**$50.00 (USD)**	Minimum a client must fund into their **own** broker account to activate copy-trading. **Not company revenue.**
`MATRIX_WIDTH`	**3**	Max direct placements per matrix position (3×3 matrix).
`MATRIX_DEPTH`	**3**	Max levels wide before spillover triggers.
`GENERATION_LEVELS`	**2–6**	Generation Bonus applies to levels 2 through 6 of a member's downline.
`MAX_LEG_CONTRIBUTION`	**40%**	Cap on how much a single referral leg may contribute toward Associate Rank Team Volume qualification.

SECTION 2

Registration Fee Allocation

Every new member registration triggers a single **$20** payment. This amount is split at the moment of registration into two top-level buckets.

Bucket	% of Fee	Amount (on $20)	Purpose
Company Operating Reserve	45%	$9.00	Non-distributable. Company revenue.
Community Rewards Pool	55%	$11.00	Feeds the 4 bonus types below.
TOTAL	**100%**	**$20.00**	**Must always reconcile to the full fee.**

Community Rewards Pool breakdown

The 55% ($11.00) is further split into four independently tracked sub-pools:

Sub-Pool	% of Fee	Amount (on $20)	Payout Type
Direct Referral Bonus	30%	$6.00	Instant, per-registration, paid to direct sponsor.
Generation Bonus (Gen 2–6)	10%	$2.00	Instant, per-registration, split across 5 generation tiers.
Rank / Team Volume Bonus	8%	$1.60	Accrues to a pool; paid only on milestone qualification.
Leadership Bonus	7%	$1.40	Accrues to a pool; paid only on leadership rank qualification.
TOTAL COMMUNITY POOL	**55%**	**$11.00**	**—**

Pseudocode

```
def allocateRegistrationFee(fee = 20.00):
    company_reserve              = fee * 0.45   # $9.00
    community_pool               = fee * 0.55   # $11.00
    direct_referral_pool         = fee * 0.30   # $6.00  -> Section 3
    generation_pool              = fee * 0.10   # $2.00  -> Section 4
    rank_pool_contribution       = fee * 0.08   # $1.60  -> Section 5 (accrues)
    leadership_pool_contribution = fee * 0.07   # $1.40  -> Section 6 (accrues)

    assert (direct_referral_pool + generation_pool
            + rank_pool_contribution
            + leadership_pool_contribution) == community_pool
    return {
        company_reserve, direct_referral_pool, generation_pool,
        rank_pool_contribution, leadership_pool_contribution
    }
```

SECTION 3

Direct Referral Bonus

```
FORMULA: direct_referral_bonus = registration_fee × 0.30
```

Paid once per new registration, directly and immediately to the Personally Enrolled Member's (PEM) direct **Sponsor**. On the standard $20 fee this equals **$6.00**.

Rules

- **Trigger event:** new member completes registration + fee payment.
- **Recipient:** the direct Sponsor only (one level up — not the whole upline).
- **Frequency:** paid once per registration event, not recurring.
- **Qualification gate:** Sponsor must be an **Active Member** (Section 9) to receive payout; if not, apply the **Compression Rule**.

Pseudocode

```
def payDirectReferralBonus(newMember, fee = 20.00):
    bonus = fee * 0.30  # $6.00
    sponsor = newMember.sponsor
    if sponsor is None:
        return  # no upline, no payout (edge case, e.g. root account)
    if not sponsor.isActiveMember():
        sponsor = getNextQualifiedUpline(sponsor)  # Compression Rule, Section 9
    if sponsor is not None:
        creditWallet(sponsor, bonus, type="DIRECT_REFERRAL_BONUS", source=newMember.id)
```

SECTION 4

Generation Bonus (Generations 2–6)

A smaller override paid to a Qualified Member for registrations occurring deeper in their downline — from Generation 2 (referrals of their referrals) through Generation 6.

Generation	Relationship	% of Fee	Amount (on $20)
Gen 2	Referrals of your Referrals	4.0%	$0.80
Gen 3	Third tier downline	2.0%	$0.40
Gen 4	Fourth tier downline	1.5%	$0.30
Gen 5	Fifth tier downline	1.5%	$0.30
Gen 6	Sixth tier downline	1.0%	$0.20
TOTAL		**10.0%**	**$2.00**

Rules

- **Generation 1** (the direct sponsor) is covered by the Direct Referral Bonus (Section 3), **NOT** this bonus.
- Each of the 5 generation levels (2–6) is evaluated independently up the tree from the new registrant.
- A member must be an **Active Member** to receive a Generation Bonus payout for their level.
- If the upline position at a given generation is NOT an Active Member, that position is skipped ("compressed") and the bonus passes to the next qualified upline position — see Section 9.

Pseudocode

```
GENERATION_RATES = {
    2: 0.040,  # $0.80
    3: 0.020,  # $0.40
    4: 0.015,  # $0.30
    5: 0.015,  # $0.30
    6: 0.010,  # $0.20
}

def payGenerationBonuses(newMember, fee = 20.00):
    uplineChain = getUplineChain(newMember, maxDepth = 6)
    # uplineChain[1] = direct sponsor (Gen 1, handled in Section 3)
    # uplineChain[2..6] = Gen 2..Gen 6
    for gen in range(2, 7):
        position = uplineChain.get(gen)
        if position is None:
            continue  # tree not deep enough yet
        recipient = position
        if not recipient.isActiveMember():
            recipient = getNextQualifiedUpline(recipient, stopAtGen = 6)  # compression
        if recipient is not None:
            amount = fee * GENERATION_RATES[gen]
            creditWallet(recipient, amount, type=f"GENERATION_BONUS_GEN{gen}",
                         source=newMember.id)
```

SECTION 5

Rank Bonus Pool & Associate Rank Milestones

```
FORMULA: rank_pool += registration_fee × 0.08   (accrues, per registration, company-wide)
```

Unlike the Direct Referral and Generation bonuses, the Rank Bonus is not paid per registration. Instead, 8% of every registration fee ($1.60) accrues into a shared pool. A member receives a one-time milestone reward the **first time** their downline Team Volume crosses an Associate Rank threshold.

Associate Rank	Personal Referrals (min.)	Team Volume (min.)	Milestone Reward*
Rank 1	3	12	$14
Rank 2	7	40	$32
Rank 3	15	130	$100
Rank 4	25	700	$280
Rank 5	35	2,500	$800
Rank 6	60	6,000	$2,400
Rank 7	100	17,000	$10,000
Rank 8	150	60,000	$20,000

> *Illustrative figures from the source document — paid once, on first qualification, subject to available pool balance and company discretion. Flag payouts for manual/admin approval if the pool balance is insufficient — **do not auto-pay past the accrued balance.**

Qualification Rules

- A member qualifies for an Associate Rank when **BOTH** the Personal Referrals count **AND** the Team Volume threshold are met (**AND** condition, not OR).
- **Leg Rule:** no single referral leg may contribute more than **40%** of the Team Volume counted toward qualification. If one leg exceeds 40% of total downline volume, only 40%-of-total worth of that leg's volume counts toward the threshold.
- Ranks are evaluated **bottom-up**: check Rank 8 down to Rank 1, and award the highest rank the member newly qualifies for (in case multiple thresholds are crossed in one update cycle).
- Once awarded, a rank milestone reward is paid **exactly once per member** ("first qualification" only).

Pseudocode

```
ASSOCIATE_RANKS = [
    # (rank_id, min_personal_referrals, min_team_volume, reward_usd)
    (1,   3,     12,    14),
    (2,   7,     40,    32),
    (3,  15,    130,   100),
    (4,  25,    700,   280),
    (5,  35,   2500,   800),
    (6,  60,   6000,  2400),
    (7, 100,  17000, 10000),
    (8, 150,  60000, 20000),
]
MAX_LEG_CONTRIBUTION = 0.40

def eligibleTeamVolume(member):
    legs = getReferralLegs(member)          # list of volumes, one per direct leg
    total = sum(legs)
    cap = total * MAX_LEG_CONTRIBUTION
    eligible = sum(min(legVolume, cap) for legVolume in legs)
    return eligible

def evaluateAssociateRank(member):
    personalReferrals = countPersonalReferrals(member)
    teamVolume = eligibleTeamVolume(member)
    qualifyingRank = None
    for (rankId, minReferrals, minVolume, reward) in reversed(ASSOCIATE_RANKS):
        if personalReferrals >= minReferrals and teamVolume >= minVolume:
            qualifyingRank = (rankId, reward)
            break
    if qualifyingRank and qualifyingRank.rankId > member.highestAssociateRank:
        if rankBonusPool.balance >= qualifyingRank.reward:
            payFromPool(rankBonusPool, member, qualifyingRank.reward, type="RANK_MILESTONE")
            member.highestAssociateRank = qualifyingRank.rankId
        else:
            flagForAdminReview(member, qualifyingRank)  # pool underfunded
```

SECTION 6

Leadership Bonus Pool & Leadership Ranks

```
FORMULA: leadership_pool += registration_fee × 0.07   (accrues, per registration, company-wide)
```

7% of every registration fee ($1.40) accrues into a Leadership Bonus Pool. Leadership Ranks sit above Associate Ranks and reward members who have developed multiple qualified leaders within their own

team. Rewards in the source document are **non-cash** (devices, trips, vehicles, housing) — model these as a **reward type/SKU**, not a currency amount.

#	Leadership Rank	Requirement	Reward Type
1	Bronze Leader	2 legs · 3 team members at Associate Rank 3	Smartphone
2	Silver Leader	2 legs · 3 team members at Associate Rank 4	Laptop
3	Gold Leader	2 legs · 3 team members at Associate Rank 5	Luxury Watch
4	Platinum Leader	2 legs · 3 team members at Associate Rank 6	International Trip
5	Diamond Leader	2 legs · 3 team members at Associate Rank 7	Luxury Vehicle
6	Crown Diamond	2 legs · 3 team members at Associate Rank 8	Home / Apartment

Qualification Rules

- **Self-qualification gate:** the member must personally already hold the Associate Rank being required of their team (e.g. a Crown Diamond candidate must already be Associate Rank 8 themselves).
- **Leg spread rule:** the 3 qualifying team members must be spread across at least 2 separate referral legs — a single leg holding all 3 does NOT qualify.
- "Team members at Associate Rank N" means: at least 3 distinct downline members (not the candidate) who have each individually reached at least Associate Rank N (Section 5).
- Leadership Ranks are evaluated in the same "highest achieved" pattern as Associate Ranks: check top-down, award once, mark as achieved permanently (see Section 9, Rank Maintenance).

Pseudocode

```
LEADERSHIP_RANKS = [
    # (rank_id, name, required_associate_rank_of_team, min_legs, min_team_members, reward_sku)
    (1, "Bronze Leader",   3, 2, 3, "SMARTPHONE"),
    (2, "Silver Leader",   4, 2, 3, "LAPTOP"),
    (3, "Gold Leader",     5, 2, 3, "LUXURY_WATCH"),
    (4, "Platinum Leader", 6, 2, 3, "INTERNATIONAL_TRIP"),
    (5, "Diamond Leader",  7, 2, 3, "LUXURY_VEHICLE"),
    (6, "Crown Diamond",   8, 2, 3, "HOME_APARTMENT"),
]

def evaluateLeadershipRank(member):
    qualifying = None
    for rank in reversed(LEADERSHIP_RANKS):
        if member.highestAssociateRank < rank.required_associate_rank_of_team:
            continue  # self-qualification gate fails
        teamAtRank = [m for m in getDownline(member)
                      if m.highestAssociateRank >= rank.required_associate_rank_of_team]
        legsUsed = distinctLegsOf(teamAtRank, member)
        if len(teamAtRank) >= rank.min_team_members and len(legsUsed) >= rank.min_legs:
            qualifying = rank
            break
    if qualifying and qualifying.rank_id > member.highestLeadershipRank:
        issueNonCashReward(member, qualifying.reward_sku, fundedBy="leadershipBonusPool")
        member.highestLeadershipRank = qualifying.rank_id
```

SECTION 7

Trading Profit Sharing

Completely separate revenue stream from the registration fee. Applies **only to realized trading profit** on a client's connected/funded broker account — never on principal, and never as a guaranteed or fixed return.

Party	Share	Basis
Client (account holder)	60%	Retains majority of realized profit on their own funded account.
Sponsor Override	10%	Paid to the client's direct Sponsor.
Company	30%	Funds trade execution, AI infra, trader oversight, risk mgmt.
TOTAL	**100%**	**Must reconcile exactly — no remainder.**

```
FORMULA: IF realized_profit > 0 → client = profit×0.60 · sponsor = profit×0.10 · company = profit×0.30
```

Rules

- **Trigger:** evaluated per settlement period (e.g. monthly) on **realized profit only** — unrealized/floating P&L does not trigger a split.
- If `realized_profit ≤ 0` (loss or breakeven) for the period: **NO split occurs**, no party is paid or charged a share.
- Sponsor Override (10%) goes to the client's direct Sponsor only — this is independent of the Sponsor's Active Member status unless business rules say otherwise (not specified in source; flag for compliance confirmation).
- This calculation **must never be presented as a fixed/guaranteed return** in any UI copy or generated statement.

Pseudocode

```
def distributeTradingProfitShare(client, periodRealizedProfit):
    if periodRealizedProfit <= 0:
        return  # no split on losses/breakeven — do not charge or pay anyone
    clientShare  = periodRealizedProfit * 0.60
    sponsorShare = periodRealizedProfit * 0.10
    companyShare = periodRealizedProfit * 0.30
    assert round(clientShare + sponsorShare + companyShare, 2) == round(periodRealizedProfit, 2)
    creditClientLedger(client, clientShare, type="TRADING_PROFIT_RETAINED")
    if client.sponsor is not None:
        creditWallet(client.sponsor, sponsorShare, type="SPONSOR_OVERRIDE", source=client.id)
    recordCompanyRevenue(companyShare, type="TRADING_PERFORMANCE_SHARE")
```

SECTION 8

Matrix Placement & Spillover Logic

New Personally Enrolled Members (PEMs) are placed into a **forced 3×3 matrix** beneath their Sponsor.

Rules

- Each matrix position may hold up to **3** direct placements (`MATRIX_WIDTH = 3`).

- The matrix extends up to **3** levels wide/deep from a given Sponsor before overflow (`MATRIX_DEPTH = 3`).
- **Spillover:** when a Sponsor's direct positions are full, additional referrals spill over to the next open position within that Sponsor's downline structure.
- Spillover placement should prioritize **balanced structure** across legs — i.e. fill the least-populated leg first — rather than stacking depth under a single line.

Pseudocode

```
def placeNewMember(newMember, sponsor):
    if sponsor.directPlacements.count < MATRIX_WIDTH:  # < 3
        sponsor.directPlacements.add(newMember)
        return sponsor
    # Sponsor's direct row is full -> spill over
    candidatePositions = getOpenPositionsInMatrix(sponsor, maxDepth = MATRIX_DEPTH)
    target = pickLeastPopulatedLeg(candidatePositions)  # balance across legs
    target.directPlacements.add(newMember)
    return target
```

SECTION 9

Active Member, Compression & Rank Maintenance Rules

9.1 — Active Member Status

A member must be an **Active Member** (good standing per the published activity policy — not in breach of the Member Agreement) to remain eligible for ongoing Generation, Rank, and Leadership Bonus payouts. Specific numeric activity thresholds are **not defined in the source document** — implement as a configurable flag/policy (e.g. `isActiveMember` boolean, or a `lastActivityDate` + inactivity window) pending compliance sign-off.

9.2 — Compression Rule

If the upline position that would normally receive a Generation Bonus is NOT an Active Member, that position is skipped ("compressed") and the bonus passes to the next qualified (Active) upline position — ensuring payouts always reach an Active, Qualified Member.

```
def getNextQualifiedUpline(position, stopAtGen = 6):
    current = position.uplineParent
    depth = 0
    while current is not None and depth < stopAtGen:
        if current.isActiveMember():
            return current
        current = current.uplineParent
        depth += 1
    return None  # no qualified upline found within compression window -> bonus is not paid
```

9.3 — Rank Maintenance

Associate and Leadership Ranks, once achieved, are **permanent for recognition/title purposes** (highest historical rank). However, where a rank unlocks a **recurring benefit** (as opposed to the one-time milestone reward already paid), continued eligibility for that recurring benefit requires maintaining Active Member status.

```
member.highestAssociateRank   # never decreases — permanent title
member.highestLeadershipRank  # never decreases — permanent title

def isEligibleForRecurringRankBenefit(member):
    return member.isActiveMember()  # title persists regardless; recurring perks gated on activi
```

SECTION 10

Reserve / Sustainability Model

Governs how each registration fee is bucketed at the system/accounting level. This is the **authoritative source-of-truth split** that Section 2 implements per-transaction.

Reserve	Source	Funding Pattern
Company Operating Reserve	45% of each fee	Non-distributable.
Direct & Generation Bonus Pool	40% of each fee	Paid out per-registration; fully funded by that same transaction (30% direct + 10% generation).
Rank Bonus Pool	8% of each fee	Accumulates across ALL registrations company-wide; balance-monitored, not guaranteed per-payout.
Leadership Bonus Pool	7% of each fee	Accumulates across ALL registrations company-wide; balance-monitored, not guaranteed per-payout.

Implementation note — pool solvency check

The Rank Bonus Pool and Leadership Bonus Pool are **shared, accumulating pools** funded by ALL registrations, not per-referral. Before paying out a Rank/Leadership milestone, the system **MUST** check `pool.balance >= reward_amount`. If insufficient, **queue/flag the payout rather than auto-paying** — the source document explicitly states these rewards are "not guaranteed" and "subject to the pool's available balance."

```
def onRegistrationFeeReceived(fee = 20.00):
    companyReserve.credit(fee * 0.45)
    directGenerationPool.credit(fee * 0.40)   # settled immediately per Sections 3 & 4
    rankBonusPool.credit(fee * 0.08)          # accumulates, Section 5
    leadershipBonusPool.credit(fee * 0.07)    # accumulates, Section 6
```

SECTION 11

Worked Examples (Test Cases)

Test Case 1 — Single Direct Referral

Input: Amaka refers one new member, Tunde, who pays the $20 registration fee.

Recipient	Bonus Type	Expected Amount
Amaka (Sponsor)	Direct Referral Bonus	$6.00
Company	Operating Reserve	$9.00
Rank + Leadership Pools	Accrued (not yet paid out)	$3.00
Generation Pool (Amaka's upline)	Accrued across Gen 2–6	$2.00
TOTAL		**$20.00**

Test Case 2 — Trading Profit Share

Input: Chidi funds his broker account with the $50 minimum. In a given month, the connected strategy realizes $200 in trading profit on his account.

Recipient	Share	Expected Amount
Chidi (Client)	60%	$120.00
Chidi's Sponsor	10%	$20.00
Company	30%	$60.00
TOTAL	**100%**	**$200.00**

Edge case: if the account instead realizes a loss that month (`periodRealizedProfit ≤ 0`), expected result is that **NO profit share is paid to any party**.

Suggested unit test skeleton

```
def test_single_direct_referral():
    result = allocateRegistrationFee(20.00)
    assert result.direct_referral_pool == 6.00
    assert result.company_reserve == 9.00
    assert result.generation_pool == 2.00
    assert (result.rank_pool_contribution
            + result.leadership_pool_contribution) == 3.00

def test_trading_profit_share_positive():
    shares = distributeTradingProfitShare(profit=200.00)
    assert shares.client  == 120.00
    assert shares.sponsor == 20.00
    assert shares.company == 60.00

def test_trading_profit_share_loss():
    shares = distributeTradingProfitShare(profit=-50.00)
    assert shares is None  # no split on loss
```

SECTION 12

Consolidated Reconciliation Check

Run these two invariants as automated sanity checks on every deploy / config change. Both must hold **exactly (to the cent)** for the plan to be internally consistent.

Check	Formula	Expected
Registration fee split	`company_reserve% + direct_referral% + generation% + rank% + leadership%`	45 + 30 + 10 + 8 + 7 = **100%**
Trading profit split	`client% + sponsor% + company%`	60 + 10 + 30 = **100%**

```
def test_reconciliation():
    fee_total = 0.45 + 0.30 + 0.10 + 0.08 + 0.07
    assert round(fee_total, 4) == 1.0000
    profit_total = 0.60 + 0.10 + 0.30
    assert round(profit_total, 4) == 1.0000
```

Not covered in this extraction

AML/KYC, data privacy (NDPR), securities-law classification, and formal Member Agreement terms are compliance/legal matters from Section 17 of the source document, not calculation logic — they are intentionally excluded here. **Confirm regulatory classification with counsel before go-live.**
